

Домашнее задание по темам 11-12

Дисциплина: DataOps

Тема: GitLab и GitLab Runner и GitLab CI/CD

Форма проверки: задание с самопроверкой. Это задание преподаватель не проверяет, но мы можем выборочно посмотреть некоторые работы на курсе.

Имя преподавателя: Дмитрий Бондарев

Примерное время выполнения: 2 часа.

Цель задания: получить практический опыт в создании полноценного CI/CD пайплайна. Научиться автоматизировать процессы проверки качества кода, тестирования и сборки Docker-образов для веб-приложения на Python.

Правила приёма работы:

1. Выполняйте задание в командах или индивидуально.
2. Разместите ссылку на выполненное задание в личном кабинете. Убедитесь, что доступ к документу открыт.

Дедлайн: рекомендуем выполнить задание в течение 7 дней после вебинара по изучаемой теме.

Описание задания:

Шаг 1: Подготовка проекта FastAPI

Создайте новый проект в GitLab. Локально создайте структуру файлов, как показано ниже.

1. main.py — основной файл вашего приложения:

```
# main.py
from fastapi import FastAPI

app = FastAPI()

@app.get("/")
def read_root():
    return {"message": "Нетология DataOps!"}

@app.get("/items/{item_id}")
def read_item(item_id: int, q: str | None = None):
```

```
return {"item_id": item_id, "q": q}
```

2. **test_main.py** — файл с тестами для вашего приложения:

```
# test_main.py
from fastapi.testclient import TestClient
from main import app

client = TestClient(app)

def test_read_root():
    response = client.get("/")
    assert response.status_code == 200
    assert response.json() == {"message": "Нетология DataOps!"}

def test_read_item():
    response = client.get("/items/5?q=somequery")
    assert response.status_code == 200
    assert response.json() == {"item_id": 5, "q": "somequery"}
```

3. **requirements.txt** — файл с зависимостями:

```
fastapi
uvicorn[standard]
pytest
requests
flake8
```

4. **`.gitignore`** — чтобы не коммитить лишнее:

```
__pycache__/
*.pyc
.venv/
```

Шаг 2: Создание Dockerfile

В корне проекта создайте `Dockerfile`, который будет описывать, как упаковать ваше FastAPI-приложение в контейнер.

```
# 1. Используем официальный образ Python
FROM python:3.10-slim

# 2. Устанавливаем рабочую директорию внутри контейнера
WORKDIR /app

# 3. Копируем файл с зависимостями
COPY requirements.txt .

# 4. Устанавливаем зависимости
RUN pip install --no-cache-dir -r requirements.txt

# 5. Копируем весь остальной код приложения
COPY . .

# 6. Указываем команду для запуска при старте контейнера
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "80"]
```

Шаг 3: Создание .gitlab-ci.yml

В корне проекта создайте файл `.gitlab-ci.yml`. Заполните его, следуя инструкциям из лекции. Ниже приведен полный пример, который вы должны понять и адаптировать.

```
# Определяем этапы нашего пайплайна
stages:
  - lint
  - test
  - build

# Задача для проверки качества кода
lint_code:
  stage: lint
  image: python:3.10-slim
  before_script:
    - pip install flake8
  script:
```

- echo "Running linter..."
- flake8 . --count --select=E9,F63,F7,F82 --show-source --statistics

Задача для запуска юнит-тестов

run_tests:

stage: test

image: python:3.10-slim

before_script:

- pip install -r requirements.txt

script:

- echo "Running tests..."
- pytest

Задача для сборки и публикации Docker-образа

build_and_push_image:

stage: build

image: docker:20.10.16

services:

- docker:20.10.16-dind

before_script:

- docker login -u \$CI_REGISTRY_USER -p \$CI_REGISTRY_PASSWORD \$CI_REGISTRY

script:

- echo "Building the Docker image..."
- IMAGE_TAG="\$CI_REGISTRY_IMAGE:\$CI_COMMIT_SHORT_SHA"
- docker build -t \$IMAGE_TAG .
- echo "Pushing image to registry: \$IMAGE_TAG"
- docker push \$IMAGE_TAG
- docker tag \$IMAGE_TAG "\$CI_REGISTRY_IMAGE:latest"
- docker push "\$CI_REGISTRY_IMAGE:latest"

Шаг 4: Запуск и проверка

- **Сделайте коммит и отправьте изменения в GitLab:**

```
`git add .`
```

```
`git commit -m "feat: Add initial FastAPI app and CI/CD pipeline"`
```

```
`git push`
```

- **Откройте ваш проект в GitLab** и перейдите в раздел **CI/CD → Pipelines**. Вы должны увидеть ваш первый запущенный пайплайн.
- **Дождитесь его завершения.** Все этапы должны быть отмечены зелеными галочками.

- **Проверьте результат.** Перейдите в раздел **Packages & Registries → Container Registry**. Вы должны увидеть опубликованный образ с двумя тегами: ``latest`` и уникальным тегом вашего коммита.

Критерии оценки (Что считается выполненным заданием)

- В репозитории присутствуют все необходимые файлы: `main.py`, `test_main.py`, `requirements.txt`, `Dockerfile`, `.gitlab-ci.yml`.
- Файл `.gitlab-ci.yml` описывает три этапа: `lint`, `test`, `build`.
- Пайплайн успешно завершается после `git push` (все задачи "зеленые").
- В Container Registry вашего проекта появился Docker-образ с двумя тегами.

Подсказки и полезные советы

- **Runner:** Убедитесь, что для вашего проекта доступен и активен GitLab Runner (общий или специфичный).
- **Логи:** Если какая-то задача падает (отмечена красным крестиком), кликните на нее и внимательно прочитайте лог выполнения. Ошибка почти всегда будет описана там.
- **Переменные:** Помните, что переменные вроде ``$CI_REGISTRY_USER`` или ``$CI_COMMIT_SHORT_SHA`` GitLab предоставляет автоматически. Их не нужно создавать вручную.

Задание со звездочкой (необязательно)

- Добавьте в `.gitlab-ci.yml`` четвертый этап ``deploy``. Задача на этом этапе должна (симулировано) "разворачивать" ваше приложение. Для этого достаточно вывести в лог сообщение вроде ``echo "Deploying image $CI_REGISTRY_IMAGE:$CI_COMMIT_SHORT_SHA to staging..."``.
- Используйте ключевое слово ``rules``, чтобы задача ``deploy`` запускалась только для ``main`` ветки.

Формат сдачи: ссылка на Google Doc с открытым доступом на чтение или на GitHub-репозиторий с файлом README.md.